# Multi-tenancy

FDDT serves several client companies from one deployment. Each company's users see **only** that company's data; the platform operator's own team (platform admins) sits outside every company.

## Roles

| Role | Company | Can |
|---|---|---|
| `user` | exactly one | submit cases/documents, track own cases |
| `reviewer_l1` | exactly one | review the company's queue; approve / reject / escalate (L1 tier) |
| `reviewer_l2` | exactly one | everything L1 can, plus escalated (L2) cases, plus the company's own **Settings › Issuer Registry** and **Settings › Risk Rules** |
| `platform_admin` | **none** | create companies; create every user and assign company + role; edit the **platform risk-rule template** new companies start from; billing & usage dashboard; processing-queue monitor; soft-delete / restore any company's case (the company never sees a deleted case, nor that it existed); **read-only, audited** support access to any company's cases, audit log and settings (and may edit a company's issuer registry/risk rules on its behalf, also audited) |

- Company roles are ranked ( `has_rank` / `hasRank` ); `platform_admin` is deliberately *not* on that ladder, so it never inherits "reviewer" powers — it cannot approve, reject, escalate, upload, create cases or generate reports.
- There is no company-level user management: **Settings › Users** is platform-admin only.
- The former single-tenant `admin` role was migrated to `platform_admin` .

## Data model

- `companies (id, name, is_active, created_at, updated_at)` . `is_active = false` suspends a company: its users can't sign in and existing tokens stop working on the next request.
- `company_id` (FK, indexed) on every tenant-owned table: `users` , `cases` , `documents` , `document_checks` , `document_page_hashes` , `cross_document_findings` , `case_actions` , `case_risk_assessments` , `risk_scores` , `risk_rules` , `risk_settings` , `issuer_registry` , `signature_references` , `signature_matches` , `case_reports` , `audit_log` .
  - NOT NULL everywhere except `users` (NULL = platform admin; a CHECK constraint enforces `role = 'platform_admin' ⇔ company_id IS NULL` ) and `audit_log` (NULL = platform-level event such as *company created*, *template edited*, *usage reconciled*, and **everything a platform admin does**: support access, *user created / updated*, *password reset*, and issuer-registry / risk-rule / threshold edits made for a company). Those rows carry `platform_admin: true` and the company in `event_data.company_id` ; only platform admins see them (in the platform log, and in that company's log when they open it). Companies and their users see nothing about platform admins. A platform admin's settings edit is visible to the company as a change (a rule's new version, new threshold values) but never names the admin: the "changed by" is left blank, and nothing labels it as a platform action.

    - Extracted fields are a column on `documents` and duplicate-detection hashes live in `document_page_hashes` — both covered by those tables' `company_id`.

- `risk_rules`, `risk_settings` (tier thresholds) and `issuer_registry` are **per company**. Rule uniqueness is `(company_id, rule_id, version)`. The issuer registry starts empty.
- `risk_rule_templates` (platform-level, **no** `company_id`) holds the default rule set. Platform admins edit it under **Platform › Rule templates**. When a company is created, every **active** template row is copied into that company's own `risk_rules` (version 1, `company_id` stamped) plus default thresholds (`seed_risk_rules(db, company_id)`). From then on the copy is the company's own: editing the template never changes an existing company — only companies created afterwards (the same "no retroactive change" principle as rule versioning). The migration seeded the template from the Default Company's current rules (38 rules — its 42 rows were 38 rules plus 4 older versions). Only `fddt_platform` has access to the table.
- `company_usage_stats (company_id, usage_date, cases_created, documents_uploaded, files_stored, storage_bytes)` — see Billing & usage.

## Tenant isolation — two independent layers

### 1. Application layer (`app/db/tenancy.py`, `app/api/auth.py`, `app/api/tenant_access.py`)

- The JWT carries `role`, `company_id` (null for platform admins), `is_platform_admin` and `ver` (see architecture/authentication.md). Every request re-reads the user from the database; if the token's company or platform flag no longer matches (user moved to another company, role changed across the platform boundary) the token is refused (401), and a suspended company's users are refused too.
- Every protected endpoint applies **two mandatory checks**: the role check (`require_company_role`, `require_reader`, `require_platform_admin`) **and** the tenant check — the request's data session is bound to the caller's company by the `get_tenant_db` dependency, so a route cannot forget it.
- Every session is in one of three states:
    - **unbound** — refuses to query or write any tenant-owned table (raises `TenantContextError`);
    - **company** — every ORM SELECT/UPDATE/DELETE touching a tenant model gets `company_id = <bound company>` added automatically (`with_loader_criteria`, including relationship loads and `session.get()`); every INSERT is stamped with that company and a row of another company is refused at flush; objects of other companies are evicted from the identity map on binding;
    - **platform** — the explicit RLS-bypassing path (below), no automatic filter.
- On top of that backstop, service functions that read tenant data take `company_id` explicitly and filter by it themselves (`load_current_rules`, `get_risk_settings`, `match_issuer`, `run_duplicate_check`, `score_case`, `pipeline_status`, `latest_assessment`, `get_case_flags`, …).
- An id that belongs to another company is a **404** — never a silent empty result and never a 403 that would confirm the id exists.
- Celery tasks are enqueued as `task.delay(document_id, company_id)` and open a session bound to that company (`app/tasks/tenant.py`), so workers are confined exactly like requests.
- Duplicate detection compares a document only against **its own company's** history (matching across companies would leak another tenant's filenames and case numbers).

### 2. Database layer — PostgreSQL Row-Level Security

The migration `a7c1e9d3f5b2_multi_tenancy` creates two login roles and the policies:

| Role | Attributes | Used for |
|---|---|---|
| `fddt_app` (`DATABASE_APP_USER`) | NOSUPERUSER **NOBYPASSRLS** | every company-scoped request and pipeline task |
| `fddt_platform` (`DATABASE_PLATFORM_USER`) | NOSUPERUSER **NOBYPASSRLS** — full access through an explicit `platform_all` policy on every table | sign-in/token identity lookup, platform-admin screens, usage reconciliation, resolving a legacy task's company |
| owner (`DATABASE_URL`) | owns the tables; needs CREATEROLE, **not** superuser | Alembic migrations and `seed.py` only |

**No role has BYPASSRLS, and no superuser is needed anywhere.** Platform access is policy-based: `CREATE POLICY platform_all ON <table> FOR ALL TO fddt_platform USING (true) WITH CHECK (true)` on each RLS table — creating a policy only needs table ownership, so this works unchanged on managed PostgreSQL (Azure Flexible Server). Verified by running the **entire** migration chain as a NOSUPERUSER/CREATEROLE owner on a fresh database and re-running the RLS suite there. (PostgreSQL 16 requires superuser even to *mention* `SUPERUSER` / `BYPASSRLS` in `ALTER ROLE`, so those attributes are set on `CREATE ROLE` only and the migration asserts afterwards that the roles really are unprivileged.)

- RLS is enabled on every tenant-owned table (plus `companies` and `company_usage_stats`) with a `tenant_isolation` policy for `fddt_app`: `USING (company_id = fddt_current_company_id()) WITH CHECK (same)`, where `fddt_current_company_id()` reads the transaction-local setting `app.current_company_id`.
- The app sets that setting at the start of **every** transaction of a company-bound session (`after_begin` hook, `set_config(..., is_local => true)`), so a pooled connection never carries one request's company into the next. No setting ⇒ no rows.
- A bug that forgets the company filter therefore still cannot read, update, delete or insert another company's rows — the database refuses. `tests/test_rls_postgres.py` proves this with raw SQL that has **no** company filter at all.
- Grants are least-privilege and also make the append-only/immutable tables read+insert-only **in the database** for both app roles: `audit_log`, `risk_rules` (versioned), `case_risk_assessments`, `case_reports`. `users` is SELECT-only for `fddt_app` (user management is platform-only).
- The platform engine is reachable only through a session explicitly bound to platform mode (`get_system_db`, `system_session()`); the default request session is unbound and then company-bound.

**Adding a tenant-owned table later:** give it `TenantScopedMixin`, and in its migration `ENABLE ROW LEVEL SECURITY`, create the `tenant_isolation` policy (`enable_tenant_rls()` in the multi-tenancy migration shows how) and grant the two app roles explicitly. There are deliberately no default privileges, so a new table is unreachable by the app until that is done.

## Platform-admin support access (audited)

A platform admin can open any company's case, case list, audit log or settings. The route resolves which company the resource belongs to through the platform session, binds the request to **that** company (so the rest of the request is confined by both layers), and writes a `platform_admin_access` audit row, e.g.:

> Platform admin ops@example.com viewed Company B / Case CASE-1A2B3C4D

**These rows are platform-only.** They are written through the platform session with `company_id = NULL` (the target company is in `event_data.company_id`), so:

- RLS hides them from every company session — a company query cannot return them even without a filter;
- every company-facing view (Audit History, its event-type filter, a case's timeline, the case report) also excludes the event type explicitly;

- company users have no indication that platform access happened.

Platform admins see every row — in the platform-level log, and in a company's log view (which shows the company's own events plus the platform-access rows about it). **Every access is recorded individually**: `PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS` defaults to `0` (a value > 0 would collapse identical reads).

Rows written before this change were moved to `company_id = NULL` by migration `c9e3a1b5d7f4` (no row removed, no content changed).

Platform admins have no cross-company case list: the case queue takes one company at a time.

## Blob storage layout

```
companies/{company_id}/cases/{case_id}/documents/{document_id}_{sha256}{ext}
companies/{company_id}/cases/{case_id}/reports/{report_id}.pdf
companies/{company_id}/cases/{case_id}/signatures/{crop_id}.png
```

`ensure_company_blob()` refuses to sign a URL under another company's prefix. Files written before the migration keep their original paths ( `{case_id}/…` , `reports/…` , `signatures/…` ) — originals are immutable and are not moved; their database rows are company-scoped like everything else.

## Billing & usage

**Platform › Billing & usage** shows one row per company (never a mixed list of cases): cases created, documents uploaded, files stored (documents + generated reports), storage added in the period, and each company's current all-time storage and file count, for this month / last month / last 30 days / this year / all time / a custom UTC date range.

- Counters live in `company_usage_stats` , one row per company per UTC day, bumped with an atomic `INSERT … ON CONFLICT DO UPDATE SET n = n + delta` **in the same transaction** that creates the case, document or report ( `app/services/usage_service.py` ). The dashboard sums a few small rows — never a COUNT/SUM over the case tables.
- Storage is the exact byte size recorded when each file was written ( `documents.file_size_bytes` , `case_reports.file_size_bytes` ). Signature crops are small derived images and are not counted.
- `reconcile_usage_stats` (Celery beat, nightly at `USAGE_RECONCILIATION_HOUR_UTC` , default 02:00 UTC) recomputes every (company, day) from the source tables and corrects any drift, writing a platform-level `usage_stats_reconciled` audit row when it changed something. **Reconcile now** on the dashboard runs the same job on demand.
- Nothing deletes cases, documents or reports today, so counters only increase. If deletion is added it must decrement them (the nightly job would otherwise correct it the next night).

## Migrating an existing single-tenant database

`alembic upgrade head` (as the owner role):

1. creates `companies` / `company_usage_stats` and one company named `DEFAULT_COMPANY_NAME` ("Default Company");
2. adds `company_id` everywhere and assigns **every** existing row to that company;
3. turns existing `admin` accounts into `platform_admin` (no company); everyone else keeps their role in the default company;
4. backfills the usage counters from the existing data;

5. creates `fddt_app` / `fddt_platform` with the configured passwords, grants, and the RLS policies (tenant isolation for the app role, `platform_all` for the platform role).

Then `c9e3a1b5d7f4` (production hardening): removes BYPASSRLS from an existing `fddt_platform` (only if it has it), adds the `platform_all` policies, moves platform-access audit rows to platform-only, creates and seeds `risk_rule_templates`, and enforces one row per check / page hash / assessment / signature match (collapsing duplicates earlier retries left: on the dev database 6 check rows, 1 page hash, 1 assessment).

The migrations need a table-owner role with **CREATEROLE**; no superuser.

`alembic downgrade` reverses it (keeping only the oldest company's rules/thresholds, since a single tenant can't hold several companies' copies).

## Tests

| Spec check | Where |
|---|---|
| Platform admin creates companies A and B and their users | `tests/test_multi_tenancy.py::tenants` fixture |
| A's reviewer_l2 sees A's cases, issuers, rules; 404 for every B resource by id (case detail/audit/file URL/signatures/reports/actions/issuer PATCH/rule history, `company_id=B` params) | `test_reviewer_l2_sees_own_company_and_gets_404_for_the_other`, `test_a_rule_history_of_another_company_is_a_404` |
| A's reviewer_l2 cannot reach Settings › Users | `test_reviewer_l2_cannot_reach_settings_users` |
| Platform admin opens B's case; a platform-only audit row is recorded (visible to platform admins, never to A or B) | `test_platform_admin_reads_any_case_and_each_access_is_audited` |
| Billing per company matches uploads; manual cross-check; drift corrected | `test_billing_dashboard_separates_companies_and_matches_uploads` |
| RLS alone blocks cross-tenant rows with no company filter in the SQL | `tests/test_rls_postgres.py` (needs `FDDT_RLS_TEST_DATABASE_URL`) |
| No BYPASSRLS anywhere; platform access via policies; platform-only audit rows invisible to the app role | `tests/test_rls_postgres.py` (`test_no_role_has_bypassrls`, `test_every_rls_table_has_the_platform_policy`, …) |
| Company users never see platform-access rows (Audit History, filter, timeline) | `tests/test_multi_tenancy.py::test_company_users_never_see_platform_admin_access` |
| Every idempotent write (page hashes, check rows, usage counters) works under the least-privilege app role | `tests/test_rls_postgres.py::test_every_upsert_the_pipeline_uses_works_under_the_app_role` |
| New company gets the template; template edits don't change existing companies | `tests/test_rule_templates.py` |
| Tenant context never leaks through PgBouncer transaction pooling | `tests/test_pgbouncer_rls.py` (needs PgBouncer) |
| Pre-migration data intact and attributed to the default company | verified against the real dev database (see the change summary) |